LECTURE 11 – SOFTWARE QUALITY ASSURANCE

Unit Testing with Code (Exam Oriented)

---

1. Unit Testing Basics

Concept

Unit testing means testing **smallest independent parts of software** such as:

- Functions
- Methods
- Classes

Each test checks **one behavior only**.

---

2. Testing Strategies with Code Examples

2.1 Test First

Write tests first, then code.

```
import unittest

class TestAdd(unittest.TestCase):
    def test_addition(self):
        self.assertEqual(add(2, 3), 5)
```

Later, developer writes:

```
def add(a, b):
    return a + b
```

Problem:

- Hard to predict all test cases initially

2.2 Test Driven Development (TDD)

Step by step:

Step 1: Write failing test

```
def test_square(self):
    self.assertEqual(square(4), 16)
```

Step 2: Write minimum code

```
def square(x):
    return x * x
```

Step 3: Refactor and repeat

Advantage:

- Code quality improves
- Clear unit boundaries

2.3 Test Last

```
def multiply(a, b):
    return a * b
```

Later testing:

```
def test_multiply(self):
    self.assertEqual(multiply(2, 3), 6)
```

Disadvantage:

- Bugs discovered late
- Poor test coverage

---

3. Python unittest Framework

Structure

```
import unittest

class MyTest(unittest.TestCase):
    def test_example(self):
        self.assertEqual(2 + 3, 5)

if __name__ == "__main__":
    unittest.main()
```

Rules:

- Test class must inherit from `unittest.TestCase`
- Test methods start with `test_`

---

4. Basic Assertions with Code

assertEqual and assertNotEqual

```
self.assertEqual(4 + 5, 9)
self.assertNotEqual(5 * 2, 11)
```

assertTrue and assertFalse

```
self.assertTrue(10 > 5)
self.assertFalse(3 > 10)
```

assertIn and assertNotIn

```
self.assertIn(3, [1, 2, 3])
self.assertNotIn(4, [1, 2, 3])
```

assertIsNone and assertIsNotNone

```
x = None
self.assertIsNone(x)

y = 5
self.assertIsNotNone(y)
```

---

5. Comparative Assertions with Code

assertAlmostEqual

Used for floating-point values.

self.assertAlmostEqual(22/7, 3.14, places=2)

Why:

- Floating values are imprecise

Numeric Comparisons

import math

self.assertGreater(math.pi, 3)
self.assertLess(3, 10)
self.assertGreaterEqual(5, 5)
self.assertLessEqual(4, 4)

Regular Expression Assertions

self.assertRegexpMatches("Software Quality Assurance", "Quality")
self.assertNotRegexpMatches("Software Testing", "Security")

---

6. Assertions for Collections

Lists

self.assertListEqual([1, 2, 3], [1, 2, 3])

Tuples

self.assertTupleEqual((2, 4, 6), (1*2, 2*2, 3*2))

Dictionaries

self.assertDictEqual({1:10, 2:20}, {2:20, 1:10})

Sets

self.assertSetEqual({1,2,3}, {3,2,1})

---

7. Skipping Tests in unittest

Skip Unconditionally

```
@unittest.skip("Skipping this test")
def test_skip(self):
    self.assertEqual(1, 2)
```

Skip If Condition True

```
@unittest.skipIf(5 > 3, "Condition true")
def test_skip_if(self):
    self.assertTrue(False)
```

Skip Unless Condition True

```
@unittest.skipUnless(5 == 5, "Condition false")
def test_skip_unless(self):
    self.assertTrue(True)
```

Expected Failure

```
@unittest.expectedFailure
def test_expected_fail(self):
    self.assertEqual(2 * 2, 5)
```

Test fails but not counted as failure.

---

8. Test Case vs Test Suite vs Test Runner

Test Case

```
class MyTest(unittest.TestCase):
    def test_add(self):
        self.assertEqual(2 + 2, 4)
```

Test Suite

```
suite = unittest.TestSuite()
suite.addTest(unittest.makeSuite(MyTest))
```

Test Runner

```
runner = unittest.TextTestRunner()
runner.run(suite)
```

---

9. Test Fixtures with setUp and tearDown

Without setUp (Bad Practice)

```
def test1(self):
    x = 10
    self.assertEqual(x, 10)

def test2(self):
    x = 10
    self.assertTrue(x > 0)
```

With setUp (Correct)

```
class TestExample(unittest.TestCase):
    def setUp(self):
        self.x = 10

    def test1(self):
        self.assertEqual(self.x, 10)
```

```
    def test2(self):
        self.assertTrue(self.x > 0)
```

---

tearDown

```
def tearDown(self):
    del self.x
```

Runs after every test.

---

10. setUpClass vs setUp

```
class Example(unittest.TestCase):

    @classmethod
    def setUpClass(cls):
        print("Runs once")

    def setUp(self):
        print("Runs before each test")

    def test1(self):
        print("Test 1")

    def test2(self):
        print("Test 2")

    @classmethod
    def tearDownClass(cls):
        print("Runs once at end")
```

Execution order:

```
setUpClass
setUp
test1
tearDown
setUp
```

test2
tearDown
tearDownClass

---

11. Module-Level Fixtures

```
def setUpModule():
    print("Module setup")

def tearDownModule():
    print("Module teardown")
```

Used for very expensive resources.

---

12. Pytest Basics with Code

Simple Test

```
def test_square():
    assert 2 * 2 == 4
```

No class required.

Multiple Tests

```
def test_add():
    assert 3 + 2 == 5

def test_sub():
    assert 5 - 2 == 3
```

---

13. Pytest Fixtures

```
import pytest

@pytest.fixture
```

```
def data():
    x = 5
    yield x
```

Used in tests:

```
def test_square(data):
    assert data * data == 25
```

Before `yield` → setup
After `yield` → teardown

---

14. Pytest Markers

Skip

```
@pytest.mark.skip(reason="Not ready")
def test_skip():
    assert False
```

Expected Failure

```
@pytest.mark.xfail
def test_fail():
    assert 2 * 2 == 5
```

Parametrize

```
@pytest.mark.parametrize("x", [1,2,3])
def test_square(x):
    assert x * x > 0
```

---

15. Fibonacci Example (Slow vs Fast)

```
def fib(n):
    old, new = 0, 1
```

```
    for i in range(n):
        old, new = new, old + new
    return old

def rfib(n):
    if n <= 1:
        return n
    return rfib(n-1) + rfib(n-2)
```

Test:

```
@pytest.mark.slow
def test_rfib():
    assert rfib(10) == 55
```

---

16. PhoneBook Case Study (Very Important)

Implementation

```
class PhoneBook:
    def __init__(self):
        self.contacts = {}

    def add(self, name, number):
        self.contacts[name] = number

    def search(self, name):
        return self.contacts[name]

    def is_consistent(self):
        for n1, num1 in self.contacts.items():
            for n2, num2 in self.contacts.items():
                if n1 != n2 and num1 == num2:
                    return False
        return True
```

Unit Tests

```
import unittest
```

```
class TestPhoneBook(unittest.TestCase):

    def setUp(self):
        self.pb = PhoneBook()

    def test_search(self):
        self.pb.add("Ali", "123")
        self.assertEqual(self.pb.search("Ali"), "123")

    def test_empty(self):
        with self.assertRaises(KeyError):
            self.pb.search("Ali")

    def test_consistent_empty(self):
        self.assertTrue(self.pb.is_consistent())

    def test_consistent_unique(self):
        self.pb.add("Ali", "123")
        self.assertTrue(self.pb.is_consistent())

    def test_inconsistent(self):
        self.pb.add("Ali", "123")
        self.pb.add("Tahir", "123")
        self.assertFalse(self.pb.is_consistent())
```
